
Prerequisites

Students must know about the following topics:

- Partition/hierarchical based clustering, K-means clustering
- Basic mathematics regarding area and volumes
- KNN, nearest neighbor search

Learning Objectives

After reading this notebook, students will be able to:

- Explain density-based clustering and its advantages over partition-based clustering.
- Exemplify terminologies such as ϵ neighborhood, density, core points, boundary points, outliers, density reachability, and connectivity.
- Describe the DBSCAN algorithm and commonly used terminologies and pros/cons.

▼ Introduction

Previously, you studied a popular iterative partition-based clustering method, *K-means clustering*, where one needs to specify the number of centroids (clusters) K explicitly. Then, the algorithm:

1. Assigns the data points to the nearest randomly initialized centroid.
2. Computes new centroids using the new assignments.
3. Reassigns data points to the nearest cluster centroids.
4. Repeats step 2 and 3 until the centroids converge.

Recall that K-means uses similarity metrics such as the Euclidean distance to measure the distance between each data point and cluster centroid.

The advantage of K-means was the ease of interpretation and implementation. However, this algorithm has limitations, such as:

- The requirement of choosing the number of centroids/clusters, K .
- No segregation of outliers from other data points.

In K-means, there is no distinction between outliers and regular data points. Those outliers are assigned to clusters though they don't belong to any. Recall, K-means's objective is to minimize the SSE of euclidean distance. Since outliers are furthest from any cluster, they tend to pull the centroids towards them.



Figure 1: K-means assigned outliers to clusters.

- Improper clustering in non-convex(non-spherical) datasets, for example: cluster shaped as ellipse, circles, moons, etc.

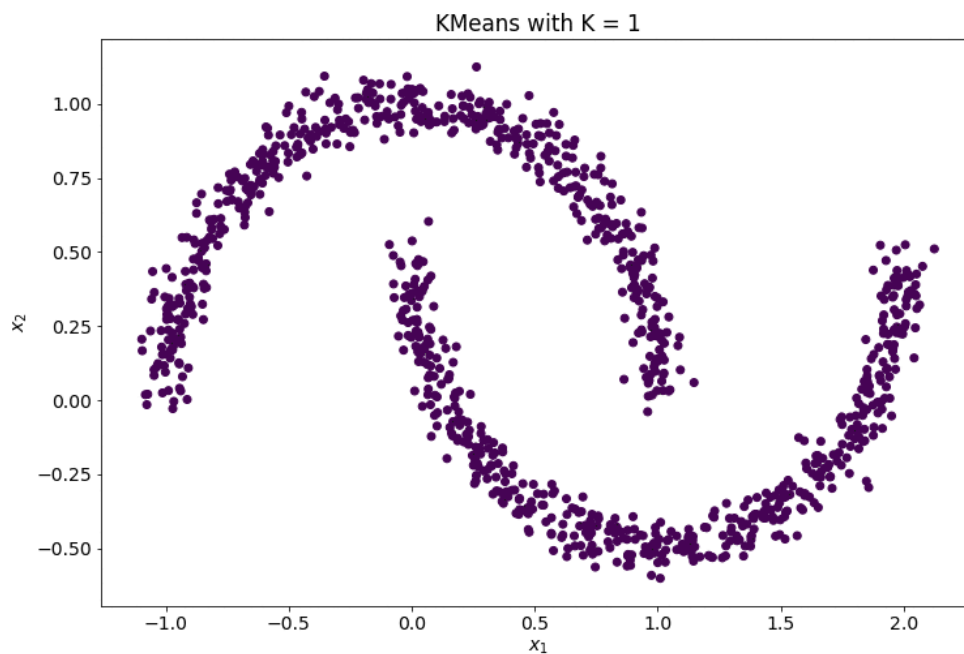


Figure 2: K-means failing to cluster non-convex(moons) 2-D dataset with varying K.

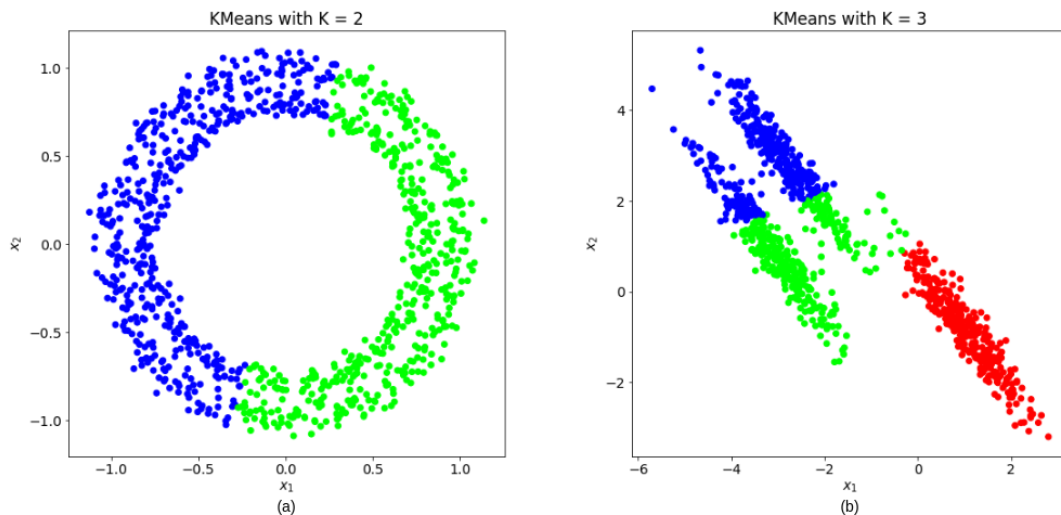


Figure 3: K-means failing to cluster 2-D (a) circles dataset | (b) ellipse dataset.

As you can see in figures 2 and 3, the K-means algorithm is unable to cluster non-convex datasets.

Using other notion than the one used in K-means for clustering, for example, hierarchical clustering methods, define clusters based on relative distances between points and overcome some limitations of K-means clustering. However, hierarchical clustering has its own drawbacks, such as:

1. Being *Computationally intensive* than K-means.
2. Explicit defining of the level at which the dendrogram is cut.

Another alternate approach for clustering data points known as **density-based clustering** is available that uses the **notion of the neighborhood and density** of data points. In this notebook, we are learning theory on density-based clustering and the most popular density-based clustering technique, DBSCAN, with its advantages/disadvantages.

▼ Density-Based Clustering

In contrast to partition and hierarchical-based clustering, density-based clustering defines clusters through the identification of a high "density" of points in space. Simply put, a cluster is a group of points that lie sufficiently close to each other in space. Density-based clustering is a nearly non-parametric method. It makes no assumptions on the *number of clusters or their distributions*, i.e., the algorithm can cluster non-convex datasets without requiring to specify the number of clusters.

Before discussing density-based clustering and techniques in-depth, we require knowledge of the neighborhood and density, so let's learn about them.

Terminologies: ϵ -neighbourhood and density

In density-based clustering, we are concerned about an arbitrary region/space around the data points and the number of data points contained within that region. One popular way to reason about the space around data points is using ϵ -neighborhood.

For dataset D of some N -dimension, the **ϵ -neighbourhood** of a data point p , denoted by $N_\epsilon(p)$, are the set of points q whose distance is at most ϵ (note : $\epsilon > 0$) from p . Mathematically,

$$N_\epsilon(p) = \{q \in D | \text{dist}(p, q) \leq \epsilon\}$$

In 2D space, a circle is the only shape with equidistant points from its center, so, ϵ -neighborhood of 2-dimensional data p is the circle with center at p with radius ϵ . Similarly, for 3D, the ϵ -neighborhood would be a sphere and N -dimension, an N -dimensional hypersphere. For obtaining such neighbourhood, euclidean distance is used as distance function, $\text{dist}(p, q)$. **Note:** Other metrics, for example, Manhattan distance, can be used for computation of neighborhood. However, it results in a square neighborhood.

In the figure below, we have an arbitrary synthetic 2-D dataset of 100 samples. The neighborhood of data points p with feature $(0.5, 0.5)$ represented by the red dot with radius $\epsilon = 0.2$ are the points that are at most distance 0.2 away from p .

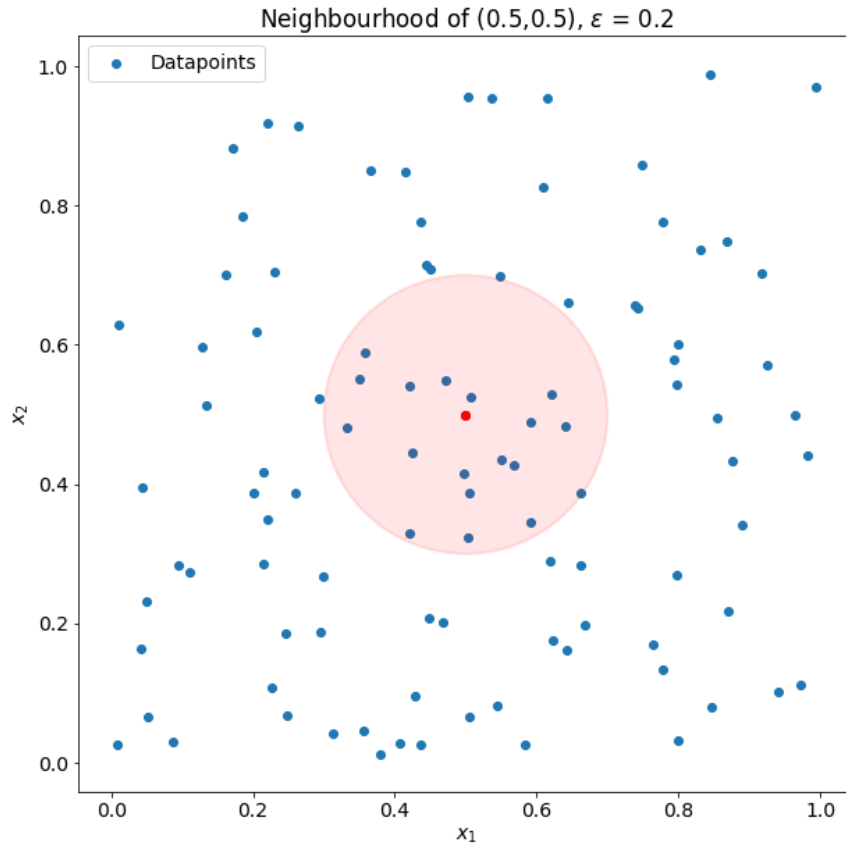


Figure 4: Neighbourhood of a datapoint with radius = 0.2.

So the neighborhood of data sample p is the red circle, which contains 19 data points. Changing the radius ϵ changes the neighborhood of data points. Now that we got the notion of

"neighborhood." Let's relate it to neighborhood "density." As you have studied in your science class, the density of an object is:

$$\text{Density} = \frac{\text{Mass}}{\text{Volume}}$$

Relating above formula for the density of neighborhood of data point p , we assume **mass** as the number of data points within the neighborhood and the **volume** as the volume of the neighborhood. However, in the 2D case, similar to *population density* of a region, instead of volume, we use area of the region in the denominator term. The neighbourhood density of data point $p(0.5, 0.5)$ of above example, is $\frac{19}{\pi 0.2^2} = 151.19$. Though the absolute value of neighborhood density is non-informative on its own, one way of using density for clustering is to calculate local density at every data point and cluster points around the area that consistently exceed certain *threshold density*. Ideally, for clustering, we want to find points with high density since *clusters are connected around a dense area in the data space and separated from each other by sparse areas*.

Another advantage of using density for clustering over partition-based clustering is *outlier detection*. *Outliers* are points having lower density(lower number of data points) around its neighborhood than other data points. Density-based clustering treats such a sparse area as noise/outlier and doesn't assign them to any cluster.

Now that we have a notion of ϵ -neighborhood and density about a particular data point, we are ready for further discussion. In this next sub-section, we are discussing the most popular density-based clustering algorithm known as DBSCAN.

▼ DBSCAN

DBSCAN, short for *Density-based spatial clustering of applications with noise*, is a popular density-based clustering algorithm that was introduced by [Martin Ester, Hans-Peter Kriegel, Jörg Sander and Xiaowei Xu in 1996](#). The core logic of DBSCAN is ϵ -neighborhood used for approximating density around data point and clustering data points around high density. Before diving into the DBSCAN's working, we require knowledge regarding the terminologies and hyperparameter of the algorithm.

DBSCAN has two **hyperparameters** that you need to select:

1. ϵ : The radius/size of the neighborhood of a point that we discussed in the above sub-section. For a point, p , all the other data points that lie within a distance ϵ are its neighbors.
2. **MinPts** : The minimum number of data points (including itself) required inside the neighbourhood to form a cluster. Note: $\text{MinPts} \leq 2$ is equivalent single link metric - hierarchical clustering, with dendrogram cut at height ϵ .

These are hyperparameters that act as constant once set, i.e., the density of clusters formed are the same throughout the dataset. With that said, let's discuss the key components/terminologies of DBSCAN:

1. **Core Points:** A point p is a core point if the neighbourhood of p contains at least MinPts.
i.e. $|N_\epsilon(p)| \geq \text{MinPts}$

Core points are the building blocks of clusters in DBSCAN. Recalling the concept of density, since the neighbourhood of all the data points are of same size (ϵ parameter being constant), core points are data points having density greater than a threshold given as:

$$\text{Threshold density} = \frac{\text{MinPts}}{\text{volume of } \epsilon - \text{neighbourhood}}$$

2. **Border Points:** A point q is a border point if the neighborhood of q contains less than MinPts, but is *reachable* from a core point p . Obviously, border points have less dense areas than core points, but these points belong to a cluster.

The authors in the original paper describe multiple types of reachability. Let's discuss them in more detail:

- **Directly density reachable**

A point q is directly density reachable from p if q falls within the neighborhood of p and p is a core point. Mathematically.

1. $q \in N_\epsilon(p)$ and
2. $|N_\epsilon(p)| \geq \text{MinPts}$

Direct density reachable condition is symmetric for a pair of core points but non-symmetric for pair of core and border points.

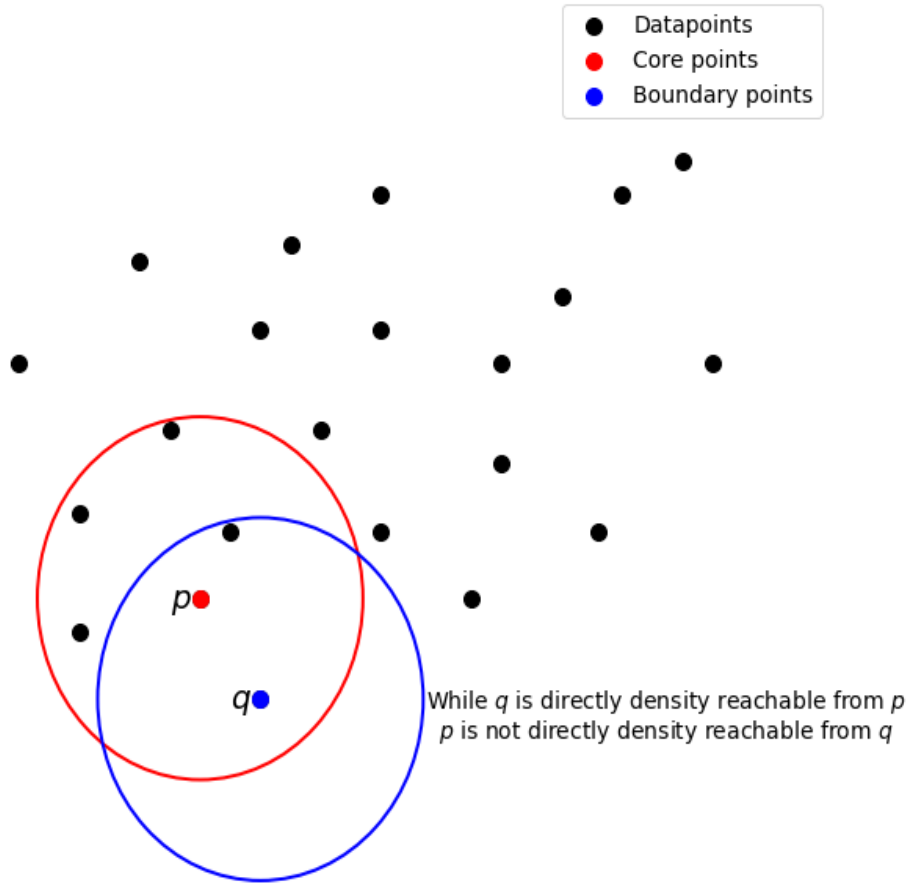


Figure 5: Directly density reachability

In the above figure, for $\text{MinPts} = 4$, q is directly density reachable from core point p as q is in the neighbourhood of p . However, q is not a core point, so the opposite doesn't hold. The red and blue colors are for denoting core and border points, respectively.

- **Density reachable**

A point q is density reachable from point p for some value of ϵ and MinPts if there exist chain of points $p_1, \dots, p_m$, $p_1 = p, p_m = q$ such that p_{i+1} is directly density reachable from p_i (Ester, 1996, p.228). It is a canonical extension of direct density reachability. While this condition is non-symmetric, it's transitive in nature.

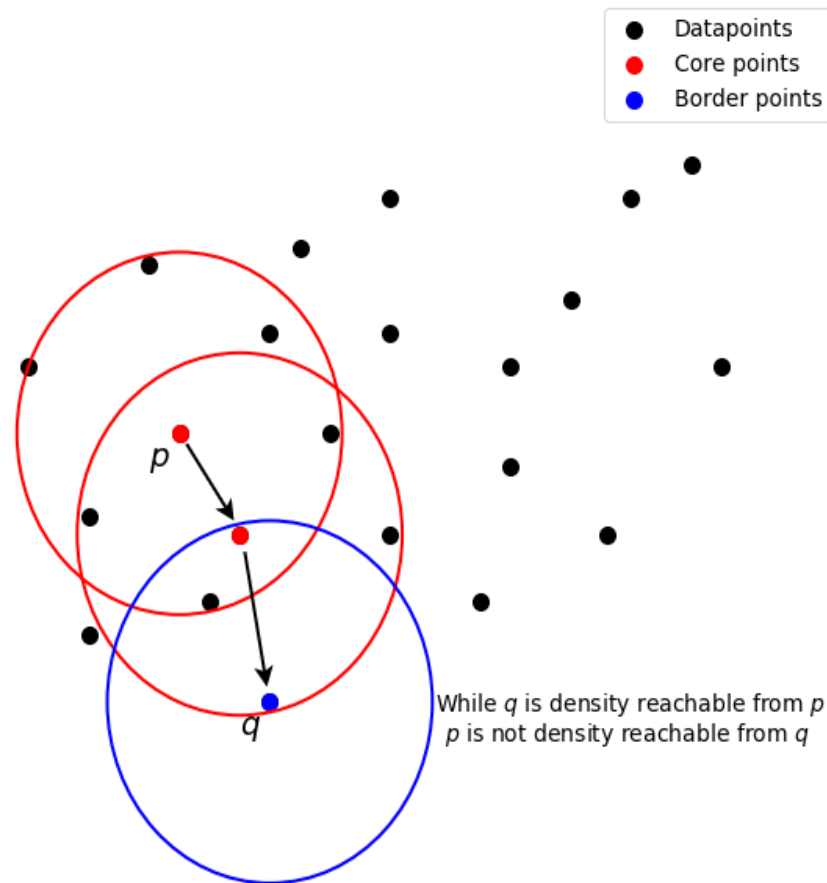


Figure 6: Density reachability

Above figure shows the case of points q being density reachable from core point p ($\text{MinPts} = 4$). Similar to directly density reachable, the opposite isn't true, i.e. p isn't density reachable from q .

Finally, let's introduce a term to describe the relation between two border points:

- **Density Connectivity**

A point q is density connected to point p for some value of ϵ and MinPts if there exists a point x such that both, p and q are density-reachable from x (Ester, 1996, p.228). Note: Density connectivity is symmetric relationship.

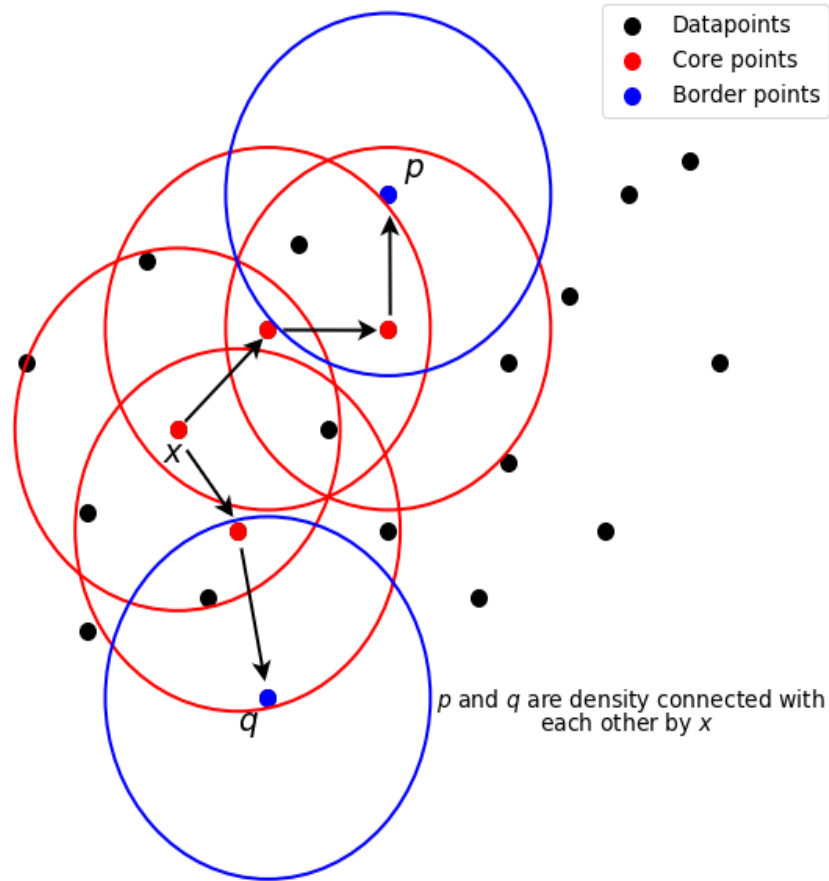


Figure 7: Density connectivity

In the above figure, for $\text{MinPts} = 4$, points p and q are both density reachable from intermediate point x . Ergo, points p and q are density connected with each other.

3. Cluster

For dataset D , a cluster C w.r.t MinPts and ϵ is a non-empty subset of point $\in D$ following conditions:

1. $\forall p, q$: if $p \in C$ and q is density-reachable from p , then $q \in C$. This condition is known as *maximality*. (Ester, 1996, p.228)
2. $\forall p, q \in C$: if p is density connected to q . This condition is known as *connectivity*. (Ester, 1996, p.228)

DBSCAN clusters any core or border point within reach of a core point under the same cluster. A cluster formation starts with a single-core point and later expands by adding other core and border points, so a cluster must at least contain MinPts . DBSCAN might

merge two clusters of different density if they are close to each other. For two clusters with a set of points S_1 and S_2 , the distance defined as:

$$\text{dist}(S_1, S_2) = \min\{\text{dist}(p, q) | p \in S_1, q \in S_2\}$$

must be greater than ϵ for separation. This expression is the same as the single linkage or MIN algorithm you studied in the agglomerative clustering. Now that we got a notion of a cluster, let's learn which points are considered outliers.

4. Noise/Outlier

For cluster $C_1, \dots, C_k$ be the clusters of dataset D w.r.t ϵ and MinPts, noise are the set of data points $\in D$ that don't belong to any clusters $C_i, i = \{1, \dots, k\}$. Mathematically,

$$\text{Noise} = \{p \in D | p \notin C_i \forall i\}$$

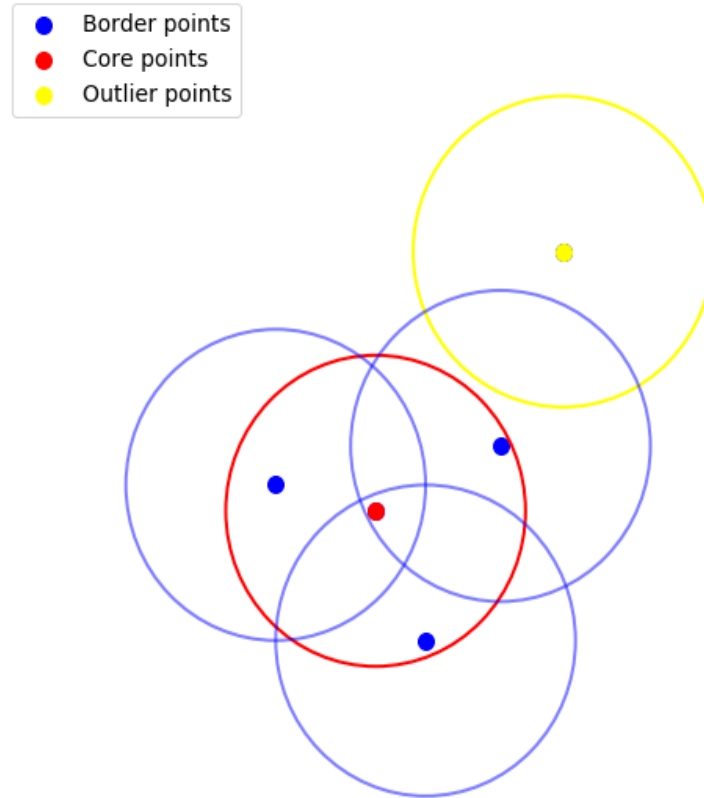


Figure 8: Core, border and outlier points

The above figure shows the classification of data points that we learned, i.e. core, border and outlier points, for an arbitrary data set with parameter $\text{MinPts} = 4$. As a refresher, the core points denoted by red consist of atleast 4 data points in its neighbourhood. The

border points denoted by blue contains ≤ 4 data points in its neighbourhood but is reachable from the core point. The border and core points falls in a cluster. Finally, the outlier denoted by yellow neither has atleast 4 data points in its neighbourhood nor is reachable from a core point.

Before moving onto the algorithm of DBSCAN, there are two key-properties to know that allow efficient computation of clusters:

1. If $p \in D$ is a core point, then set S of points that are density reachable from p w.r.t MinPts and ϵ , falls under a cluster.
2. For a cluster $C \in D$, each point is density reachable from **any** of the core point of C . Meaning, cluster C contains points that are density reachable from an arbitrary core point p of C .

Therefore, a cluster formation is determined by any of its core points. Computation of all the core points isn't required.

Finally, Let's discuss the working procedure of DBSCAN:

Algorithm

Starting with a dataset:

1. Arbitrarily choose a data point that not assigned to a cluster or classified as noise/outlier.
2. Compute the ϵ -neighborhood of the selected data point and determine if it's a core point.
 - If true, classify it as core point and assign the set of data points in its ϵ -neighborhood (directly density reachable) in a single cluster.
 - If false and unassigned to a cluster, classify it as an outlier point.
3. Next, add all the density reachable points from the selected data point to the same cluster. i.e., repeat the procedure from step 2 for all the point density reachable from initially selected point, until all the data points of a cluster are processed.

Note: A previously classified outlier can be added to a cluster, and the designation changes from outlier to border point.

4. Repeat from step 1 until all points are in a cluster or classified as noise.

That's all the steps for clustering a dataset using DBSCAN. Now, let's see DBSCAN in action with $\text{minPts} = 4$ for an arbitrary synthetic dataset.



Figure 9: DBSCAN algorithm visualization on synthetic dataset

In the above animation, the black points represent data points unassigned to any cluster. The circles's color(neighborhood color) denotes the type of data point - red = core point, blue = border point, and yellow = outlier(cross represent outlier points).

Notice that the top left data point of the green cluster was reassigned from outlier to a border point. The final output of DBSCAN for this dataset was, two clusters orange and green with six and seven data samples respectively, and four outliers.

Before moving onto the pros and cons of DBSCAN, let's briefly go through the pseudo-code of DBSCAN and discuss its run-time complexity.

PSEUDO-CODE

This sub-section contains the pseudo-code for the DBSCAN algorithm and `range_query` function for finding the distance of neighbors (Schubert, 2017). Remember DBSCAN takes

dataset (DB), distance function (dist_func, usually Euclidean distance), the radius of the neighborhood (eps) and required points in the neighborhood (min_pts) as input parameters. Note: All the data points are initially labeled "undefined".

```
DBSCAN(DB, dist_func, eps, min_pts)

C = 0                                     # Cluster counter

for each point P in dataset DB:
    if label(P) != undefined:
        continue                         #Previously processed in inner loop
    Neighbors N = range_query(DB, dist_func, P, eps)  # Find neighbors
    if |N| < min_pts:                     # Density check
        label(P) = Noise                 # Label as Noise
        continue

    C = C + 1                             #Next cluster label
    label(P) = C                          #Label initial point
    SeedSet S = N \ {P}                  #Neighbors to expand

    for each point Q in S:                #Process every seed point Q
        if label(Q) == Noise:
            label(Q) = C                  #Change Noise to border point
        if label(Q) != undefined:
            continue                      #Previously processed (e.g., border point)

        label(Q) = C                      # Label neighbor
        Neighbors N = range_query(DB, dist_func, Q, eps) #Find neighbors
        if |N| >= min_pts:                #Density check (if Q is a core point)
            S = S ∪ N                     #Add new neighbors to seed set
```

Pseudo-code for range_query

The range_query function finds the neighbors in the ϵ -neighborhood of a given data point. The pseudo-code is given as:

```
range_query(DB, dist_func, Q, eps):
    Neighbors N = empty list
    for each point P in dataset DB:      # Scan all points in the database
        if dist_func(Q, P) <= eps:      # Compute distance and check epsilon
            N = N ∪ {P}                  # Add to result
    return N
```

From the pseudo-code, you might notice that the range_query function runs twice for the DBSCAN algorithm per iteration of point P . Only when the label of data points are undefined

neighborhood region query execution takes place. Execution of this query on a data point labels it as an outlier or cluster number. Finally, the relabeling of a point only occurs when noise label changed to cluster label. Thus, the `range_query` function executes precisely once for each data point of the dataset. Consequently, the DBSCAN's loop executes at most once for each data point.

Runtime Complexity

Let's compute the runtime complexity of the DBSCAN algorithm. Since the algorithm runs once per data point, we get the runtime complexity of $O(n \cdot Q)$, where Q is the runtime complexity of `range_query` function.

In the original paper by Ester M. et al., for efficient region queries, the authors used spatial data access methods(indexing structure) known as R*-trees resulting in runtime complexity $Q = O(\log n)$. These spatial access methods are generally tree data structures used to store data indexes efficiently. For the range query, these methods give the specific range of coordinates to find the neighbors and their distance. Note: Scikit-learn uses k-d trees and ball trees for spatial data access. Using a spatial access method, the **overall average runtime complexity** of the DBSCAN algorithm in the best case is $O(n \log n)$. However, without spatial access methods, or when all the data points are within distance less than ϵ , the **worst case runtime** becomes $O(n^2)$.

Result of clustering using DBSCAN

Now that we discussed the algorithm, let's look at the clustering of non-convex dataset performed using DBSCAN:

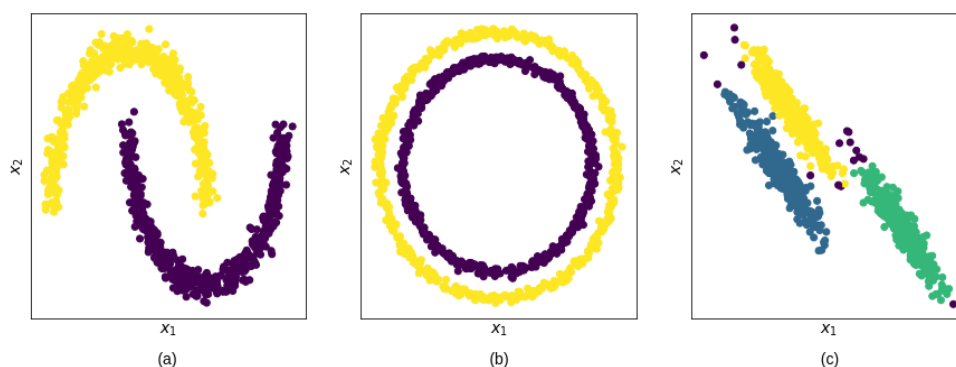


Figure 10: Outcome of clustering using DBSCAN on (a) moon | (b) circle | (c) ellipse dataset

Look at how accurately different types of datasets are clustered by using the DBSCAN algorithm. Since DBSCAN uses the notion of density, it tends to cluster highly dense regions of a dataset of any shape. DBSCAN performs better than K-means for the non-convex dataset without the need to set the number of clusters.

At last, let's discuss the pros and cons of DBSCAN.

Pros and Cons

Every algorithm has its pros and cons, so does DBSCAN.

Pros

Advantages of DBSCAN are as follows:

- Doesn't require specification of the number of clusters — inference of the number of clusters from the provided data.
- Ability to cluster data of arbitrary shapes, even those surrounded by other clusters.
- The inherent concept of outliers allows it to find clusters and separate outliers in noisy real-world data.

Cons

Limitations of DBSCAN are as follows:

- Though there is a heuristic method to select optimal values for MinPts and ϵ , setting the values requires domain knowledge. Variation in those parameters profoundly affects the clustering performance.
- Curse of dimensionality is present for finding the distance measure, usually euclidean distance.

Border points might be reachable from more than one cluster and assigned to either one, depending on the data processing order.

- DBSCAN assumes constant density of clusters throughout the dataset, (constant MinPts and ϵ). So, DBSCAN cannot cluster datasets with large variations in densities.

We have reached the end of this notebook, comprehending density-based clustering and DBSCAN. In the next chapter, you are learning another density-based algorithm known as OPTICS.

Key-Takeaways

The key points to remember from this notebook are:

1. In contrast to partition and hierarchical based clustering, density-based clustering defines clusters and outliers through the notion of neighborhood and density of points in space. In simple terms:
 - Clusters are a group of points connected around a dense area in the data space and separated by sparse regions.
 - Outliers are points present in sparse areas.

2. The ϵ -neighborhood of a data point is the set of points whose distance is at most ϵ far from that point, while the density is a ratio of the number of data points within the neighborhood and volume of the neighborhood.
3. DBSCAN, a density-based clustering algorithm, *clusters non-convex dataset with no explicit assumption on the number of clusters*, having parameters:
 - ϵ : The radius/size of the neighborhood of a point.
 - MinPts: The minimum number of data points (including itself) required inside the neighborhood to form a cluster.
4. In DBSCAN, data points are classified as core, border, and outlier points.
 - Core points: points with MinPts data points in its neighborhood.
 - Border points: points with less than MinPts data points in its neighborhood but "reachable" from the core point.
 - Outlier: less than MinPts data points in its neighborhood, isn't reachable from a core point.
5. DBSCAN algorithm's loop runs at least once per point in the data set. Ergo, the runtime complexity is $O(n \log n)$ (best case) to $O(n^2)$ (worst case).
6. DBSCAN is sensitive to its parameter and fails to cluster dataset with significant variation in the cluster's density.

References

- **Papers**
 - Ester M., et al., (1996), A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise, KDD-96 Proceedings, Retrieved from : <https://www.aaai.org/Papers/KDD/1996/KDD96-037.pdf>
 - Schubert E., et al., (2017, July), DBSCAN Revisited, Revisited: Why and How You Should (Still) Use DBSCAN, ACM Transaction on Database Systems, Retrieved from: <https://dl.acm.org/doi/10.1145/3068335>

Additional material

- Website

- [Visualizing DBSCAN Clustering](#) Interactive implementation of DBSCAN -- Don't miss this!
- Books
 - Longley P. A., Goodchild M. F., Maguire D. J., Rhind D. W., (2005), New Developments in Geographical Information Systems: Principles, Techniques, Management and Applications.
 - Study chapter 27 Spatial access methods to learn more about tree data structures.